Trabajo Tutelado AII:

Infraestructura declarativa reproducible usando OpenTofu y NixOS

Autores:
Mario Lamas Angeriz (m.lamasa@udc.es)
Alexandre Borrazás Mancebo (alexandre.bmancebo@udc.es)

Repositorio en Github

Índice

1. Propuesta **2**

2. Objetivos **2**

3. Tecnologías usadas **2**

4. Diagrama de red **3**

5. Estructura del proyecto **4**

6. Desarrollo del proyecto **5**
6.1. Configuraciones iniciales . . . 5
6.2. Declaración de los recursos . . . 5
6.3. Configuración con nixos-anywhere . . . 6

7. Gestión de cambios con Colmena **6**
7.1. Integración con el flake . . . 6
7.2. Flujo de trabajo habitual . . . 7
7.3. Ventajas operativas . . . 8
7.4. Configuración SSH para Colmena . . . 8

8. Configuración de NixOS **8**
8.1. Estructura del flake . . . 8
8.2. Módulos compartidos . . . 8
8.2.1. common.nix . . . 9
8.2.2. users.nix . . . 9
8.2.3. ssh.nix . . . 9
8.2.4. fail2ban.nix . . . 9
8.2.5. utils.nix y neovim.nix . . . 9
8.3. Configuración de hosts . . . 9

9. Servicios desplegados **9**
9.1. Caddy (proxy inverso) . . . 9
9.2. DDNS (ddns.nix) . . . 10
9.3. Workers con Nginx (nginx_worker.nix) . . . 10
9.4. Docker (docker.nix) . . . 10
9.5. Monitorización: Prometheus, Node Exporter y Grafana . . . 10

10. Ventajas y limitaciones **11**
10.1. Ventajas . . . 11
10.2. Limitaciones y desafíos . . . 11

1. Propuesta

Aprender más sobre **Infrastructure as Code** (**IaC**) usando herramientas como **OpenTofu** que permite describir y desplegar infraestructura (máquinas virtuales, redes, almacenamiento, etc.) de forma declarativa, mientras que **NixOS** (tenemos experiencia previa) proporciona un sistema operativo completamente reproducible cuya configuración también se define mediante código.

Este trabajo propone explorar la integración de ambas tecnologías para crear una infraestructura completamente declarativa, donde tanto la creación de recursos como la configuración del sistema operativo estén definidas en repositorios versionados (`git`).

2. Objetivos

El objetivo principal del trabajo es diseñar e implementar una plataforma de infraestructura reproducible basada en principios de **Infrastructure as Code**.

- Estudiar el paradigma de infraestructura declarativa.
- Analizar el funcionamiento de **OpenTofu** para la provisión de recursos.
- Analizar el modelo de configuración declarativa de **NixOS**.
- Diseñar una arquitectura donde **OpenTofu** gestione la infraestructura y **NixOS** configure los sistemas.
- Implementar un entorno reproducible donde la infraestructura pueda desplegarse automáticamente a partir del código.
- Evaluar las ventajas y limitaciones del enfoque propuesto.

3. Tecnologías usadas

Para la realización de la practica decidimos emplear las siguientes tecnologías:

- **Proxmox VE**: plataforma de virtualización open-source que utilizamos como hipervisor para la gestión de nuestras máquinas virtuales.
- **OpenTofu**: fork de Terraform open-source, herramienta de IaC que empleamos para la delcaración y asiagnación de recursos de nuestras máquinas virtuales.
- **NixOS**: distribución de Linux que emplea el gestor de paquetes Nix, gracias a su configuración delcarativa nos permite desplegar en las máquinas virtuales todo tipo de servicios de forma sencilla y automatizada.
- **nixos-anywhere**: herramienta de instalación de NixOS, nos permite realizar la instalación del SO de forma remota mediante `ssh` y de forma completamente automatizada.
- **Colmena**: tras la primer build de los sistemas de NixOS Colmena nos permite realizar cambios sobre el SO (mientras este se ejecuta) de forma remota y sencilla sin falta de reinstalar.
- **agenix**: paquete que permite la gestion de secretos mediante encriptación `age`, de esta forma podemos mantener registro de claves y compartirlas entre máquinas sin miedo a exponernos.

- **Docker**: mediante la configuración en NixOS, podemos desplegar servicios sobre Docker en boot lanzandolos de forma automatizada.

4. Diagrama de red

Para comenzar el proyecto decidimos que lo primero de todo era deifinir la estructura de red, y equipos/contenedores a crear.

Como se puede ver en la Figura 1, se creo una red **DMZ** que conectase con el router para poder aislar las máquinas y reproducir de forma más fiable una red empresarial. La red **LAN** es la red Wi-Fi del router y la **WAN** representa internet. El router realmente está virtualizado al igual que el resto de contenedores y VM's. Hemos redireccionado el puerto `3478` del router para que permita la conexión con la vpn que montamos. La redireccion del puerto `80/443` hacia la máquina con `caddy` permite el acceso externo a la infraestructura.

El resto de las máquinas de la red estan aisladas frente accesos no permitidos, ṡolamente acceptan conexiones `ssh` con certificado (se introducieron dentro de las VM nuestros certificados para un acceso más seguro y rápido).

Una vez con esto establecido podíamos centrarnos en el desarrollo del IaC.

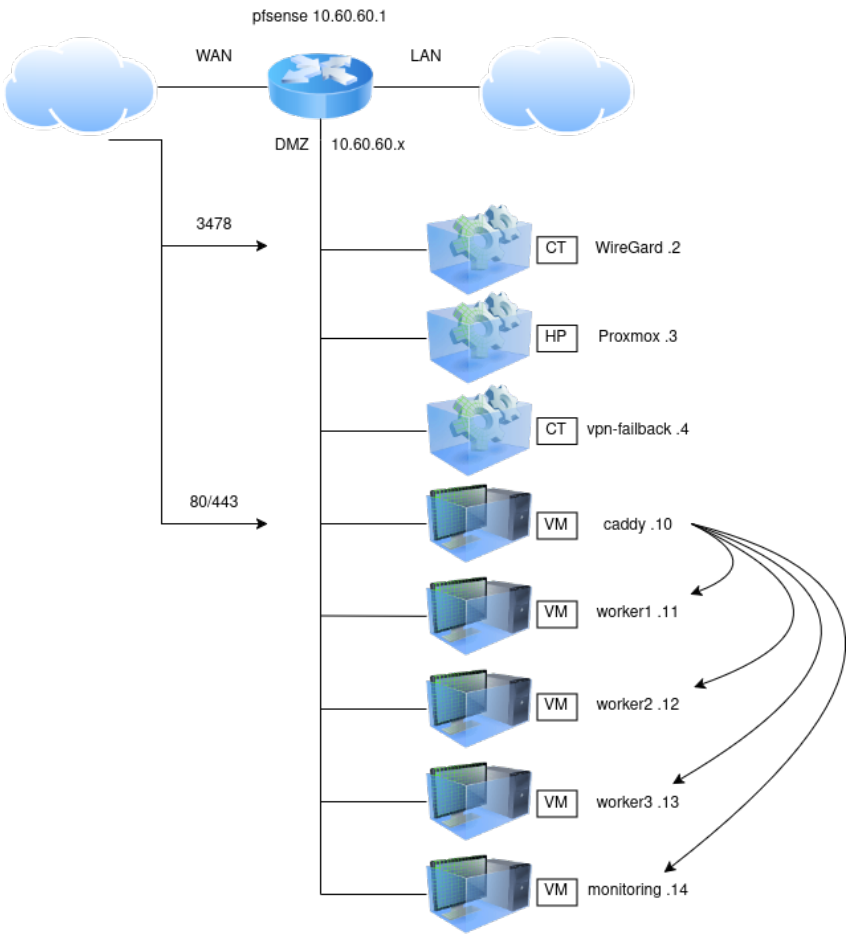

Figura 1: Esquema de red

5. Estructura del proyecto

Organizamos el proyecto en 2 carpetas:

```
.
├── nixos
│   ├── first-deploy.sh
│   ├── flake.lock
│   ├── flake.nix
│   ├── hosts
│   │   ├── caddy
│   │   │   ├── default.nix
│   │   │   ├── disk-config.nix
│   │   │   └── hardware-configuration.nix
│   │   ├── monitoring
│   │   │   └── ...
│   │   ├── worker1
│   │   │   └── ...
│   │   ├── worker2
│   │   │   └── ...
│   │   └── worker3
│   │       └── ...
│   ├── modules
│   │   ├── caddy.nix
│   │   ├── common.nix
│   │   ├── ddns.nix
│   │   ├── docker.nix
│   │   ├── fail2ban.nix
│   │   ├── grafana.nix
│   │   ├── neovim.nix
│   │   ├── nginx-indexes
│   │   │   └── ...
│   │   ├── nginx_worker.nix
│   │   ├── node_exporter.nix
│   │   ├── prometheus.nix
│   │   ├── ssh.nix
│   │   ├── users.nix
│   │   └── utils.nix
│   └── secrets
│       ├── cf-token.age
│       └── secrets.nix
└── terraform
    ├── main.tf
    ├── tofu.sh
    └── variables.tf
```

- `nixos`: dentro de esta carpeta se guardan todos los archivos de configuración relacionados con los equipos.
- `terraform`: aqui mantenemos la declaración de asignación de recursos a cada máquina.

6. Desarrollo del proyecto

6.1. Configuraciones iniciales

Para comezar se realizó la configuración de un router **pfsense** virtual que nos permitiese una buena gestión de las redes y monitoriación. Como siguiente paso configuramos un acceso remoto mediante **TailScale** como backup, pues este solo permite el acceso de uno de nosotros a la red. Montamos entonces otra VPN, **WireGuard**, que nos permite el acceso remoto a la red a los 2. Ademas el TailScale casi se monta solo por lo que queriamos hacer el WireGuard de 0 como una experiencia más real y segura.

6.2. Declaración de los recursos

Para que OpenTofu pueda interactuar con el hipervisor, es necesario configurar el proveedor de Proxmox en el archivo `main.tf`. La autenticación se realiza mediante un **API Token** para evitar el uso de la contraseña de `root`.

```
provider "proxmox" {
  endpoint  = var.proxmox_api_url
  api_token = var.proxmox_api_token
  insecure  = true
}

resource "proxmox_virtual_machine" "nixos_vm" {
  count       = 3
  name        = "worker-${count.index}"
  target_node = "pve"
  clone {
    vm_id = var.debian_template_id
  }

  network_device {
    bridge = "vmbr1" # DMZ
  }

  cpu {
    cores = 2
  }

  memory {
    dedicated = 2048
  }
}
```

Para automatizar este proceso, utilizamos un script `tofu.sh` que carga las variables de entorno desde un archivo `.env`, facilitando el despliegue sin exponer credenciales en el código fuente. El uso de una plantilla **Debian** con **Cloud-init** es clave: nos permite inyectar nuestras claves SSH públicas de forma automática, dejando la máquina lista para que NixOS tome el control.

6.3. Configuración con nixos-anywhere

Una vez que OpenTofu ha levantado las máquinas virtuales con Debian, utilizamos **nixos-anywhere** para realizar la transición a NixOS. Este proceso es crítico para la naturaleza declarativa del proyecto, ya que permite instalar NixOS en un sistema existente de forma remota.

El flujo de trabajo que seguimos es:

1. **Generación del hardware-config**: Nos conectamos a la VM recién creada para obtener la configuración específica de los controladores de Proxmox (`virtio`, `scsi`).
2. **Particionado con Disko**: Definimos el esquema de particiones en un archivo `disk-config.nix`. `nixos-anywhere` utiliza esta herramienta para formatear el disco de la VM Debian en caliente.
3. **Instalación kexec**: Se lanza un kernel de NixOS en memoria que sobrescribe el sistema Debian.

El comando utilizado en nuestro script `first-deploy.sh` es:

```
nix run github:nix-community/nixos-anywhere -- \
  --flake .#hostname \
  template@ip-de-la-vm \
  -i ~/.ssh/id_rsa
```

Posteriormente, cualquier cambio en la configuración (servicios, usuarios, Docker) se gestiona con **Colmena**, lo que permite aplicar cambios en segundos sin reiniciar la máquina ni reinstalar el sistema.

7. Gestión de cambios con Colmena

Una vez realizado el despliegue inicial con `nixos-anywhere`, la herramienta encargada de gestionar el ciclo de vida de la configuración es **Colmena**. A diferencia de `nixos-anywhere`, que instala el sistema desde cero, Colmena aplica cambios incrementales sobre máquinas ya en ejecución mediante SSH, sin necesidad de reiniciar ni reinstalar.

7.1. Integración con el flake

Colmena lee su configuración directamente del output `colmena` del `flake.nix`. Cada host se define con su dirección IP (`targetHost`), usuario de despliegue (`targetUser`) y su configuración NixOS, que es exactamente la misma que se usa en `nixosConfigurations`. Esto garantiza que no existe divergencia entre el sistema instalado y el sistema gestionado.

En el archivo `flake.nix` la sección de Colmena tiene la siguiente forma:

```
colmenaHive = colmena.lib.makeHive self.outputs.colmena;
    colmena = {
      meta = {
        nixpkgs = import nixpkgs { inherit system; };
        specialArgs = { inherit disko inputs; };
      };

      caddy = { name, nodes, pkgs, ... }: {
        deployment = {
```

```
      targetHost = "10.60.60.10";
      targetUser = builtins.getEnv "USER";

      # You can build on the target machine
      # buildOnTarget = true;
    };
    imports = caddyModules;
  };

  worker1 = { name, nodes, pkgs, ...}: {
    deployment = {
      targetHost = "10.60.60.11";
      targetUser = builtins.getEnv "USER";
    };
    imports = worker1Modules;
  };

  # worker2, worker3 ...

  monitoring = { name, nodes, pkgs, ...}: {
      deployment = {
          targetHost = "10.60.60.14";
          targetUser = builtins.getEnv "USER";
      };
      imports = monitoringModules;
  };

};
```

7.2. Flujo de trabajo habitual

El flujo típico al modificar la configuración de uno o varios hosts es el siguiente:

1. Se edita el módulo o archivo de host correspondiente en el repositorio.
2. Se verifica que la configuración compila correctamente sin desplegar:

   `nix run github:zhaofengli/colmena -- build`
3. Si la build es exitosa, se despliega el cambio. Para aplicarlo a un único host:

   `nix run github:zhaofengli/colmena -- apply --on caddy --impure`

   Para aplicarlo a toda la infraestructura de forma simultánea:

   `nix run github:zhaofengli/colmena -- apply --impure`
4. Colmena construye la nueva generación del sistema en la máquina local (o en el propio host si se configura despliegue remoto), la transfiere por SSH y ejecuta `nixos-rebuild switch`, activando los cambios en caliente.

7.3. Ventajas operativas

El uso de Colmena frente a otros enfoques de despliegue aporta varias ventajas prácticas en nuestro contexto:

- **Despliegue paralelo**: Colmena puede aplicar cambios a múltiples hosts en paralelo, reduciendo el tiempo total de actualización de la infraestructura completa.
- **SetUp sencillo**: no requiere instalar ningún agente en los hosts destino; únicamente necesita acceso SSH con el usuario y clave configurados en `deployment`.
- **Consistencia**: aplicar la misma configuración varias veces produce siempre el mismo resultado, sin efectos secundarios acumulativos.

7.4. Configuración SSH para Colmena

Para que Colmena pueda conectarse a cada host es necesario añadir una entrada en `~/.ssh/config` por cada máquina gestionada:

```
Host 10.60.60.10
  IdentityFile ~/.ssh/id_ed25519
Host 10.60.60.11
  IdentityFile ~/.ssh/id_ed25519
Host 10.60.60.12
  IdentityFile ~/.ssh/id_ed25519
Host 10.60.60.13
  IdentityFile ~/.ssh/id_ed25519
Host 10.60.60.14
  IdentityFile ~/.ssh/id_ed25519
```

De este modo Colmena selecciona automáticamente la clave correcta sin intervención manual durante el despliegue.

8. Configuración de NixOS

8.1. Estructura del flake

El punto de entrada de toda la configuración NixOS es el archivo `flake.nix`, que actúa como manifiesto del proyecto: declara las dependencias externas (inputs) y expone las configuraciones de cada host (outputs). Entre los inputs más relevantes se encuentran `nixpkgs` (el repositorio de paquetes de NixOS), `agenix` (gestión de secretos), `disko` (particionado declarativo) y `colmena` (despliegue remoto).

Cada host queda registrado tanto en el output `nixosConfigurations` (usado por `nixos-anywhere` en el primer despliegue) como en el output `colmena` (usado para actualizaciones posteriores). Esto garantiza que la misma definición de sistema se emplee independientemente del mecanismo de despliegue.

8.2. Módulos compartidos

Para evitar la duplicación de configuración entre hosts, extraemos toda la lógica reutilizable en módulos bajo `nixos/modules/`. Cada host importa el subconjunto de módulos que necesita desde su `default.nix`.

8.2.1. common.nix

Define la base común a todas las máquinas: zona horaria, locale, bootloader (**GRUB** en modo BIOS para compatibilidad con Proxmox), y el conjunto mínimo de paquetes del sistema (`git`, `htop`, `curl`, etc.). También habilita `nix.settings.experimental-features` para soportar flakes.

8.2.2. users.nix

Declara los usuarios del sistema de forma centralizada. Las claves SSH públicas de los administradores se incluyen directamente en la configuración, de modo que cualquier máquina que importe este módulo acepta las mismas identidades desde el primer arranque. Las contraseñas se deshabilitan forzando el acceso exclusivamente por certificado.

8.2.3. ssh.nix

Endurece la configuración del demonio SSH: deshabilita el login con contraseña (`PasswordAuthentication = false`). Combinado con `fail2ban`, forman una firme defensa frente a accesos remotos no autorizados en cada máquina.

8.2.4. fail2ban.nix

Habilita el servicio `fail2ban` con una jail para SSH que banea automáticamente direcciones IP tras varios intentos de autenticación fallidos. La configuración de este módulo es común a todos los hosts.

8.2.5. utils.nix y neovim.nix

Agrupan herramientas de administración del sistema (`tmux`, `ripgrep`, `jq`, `wget`, etc.) y la configuración personalizada de **Neovim** respectivamente, disponibles en todas las máquinas para facilitar el trabajo de los administradores.

8.3. Configuración de hosts

Cada directorio bajo `nixos/hosts/<hostname>/` contiene tres archivos:

- `hardware-configuration.nix`: generado a partir de la plantilla Debian; captura los módulos del kernel necesarios para el hardware virtualizado de Proxmox (`virtio_pci`, `virtio_scsi`, etc.).
- `disk-config.nix`: define el esquema de particiones mediante `disko`. Usamos una tabla **GPT** con una partición de arranque BIOS y una partición raíz `ext4`.
- `default.nix`: configuración específica del host; importa los módulos necesarios, asigna el hostname y declara los servicios particulares de esa máquina.

9. Servicios desplegados

9.1. Caddy (proxy inverso)

La máquina `caddy` actúa como punto de entrada a toda la infraestructura. El módulo `caddy.nix` habilita el servicio `services.caddy` de NixOS apuntando a un `Caddyfile` personalizado ubicado en `modules/caddy-config/Caddyfile`. **Caddy** gestiona automáticamente los certificados TLS

mediante **ACME/Let's Encrypt**, usando el token de **Cloudflare** (almacenado como secreto `age`) para el challenge **DNS-01**, lo que permite obtener certificados incluso sin exponer el puerto `80` directamente.

El `Caddyfile` define los virtual hosts necesarios para enrutar el tráfico entrante hacia los servicios internos de la red DMZ mediante proxy inverso, añadiendo cabeceras de seguridad y terminación TLS en este punto.

9.2. DDNS (ddns.nix)

Dado que la IP pública del router puede cambiar, el módulo `ddns.nix` configura un servicio de **DNS dinámico** mediante la API de Cloudflare. Se despliega como un `systemd.services` periódico que comprueba la IP pública actual y actualiza el registro DNS correspondiente en caso de cambio, usando de nuevo el token de Cloudflare gestionado por `agenix`.

9.3. Workers con Nginx (nginx_worker.nix)

Los hosts `worker1`, `worker2` y `worker3` son máquinas de propósito general que ejecutan servicios sobre Docker. El módulo `nginx_worker.nix` levanta un servidor **Nginx** en cada worker que sirve una página de índice personalizada (los archivos `index1.html`, `index2.html` e `index3.html` ubicados en `modules/nginx-indexes/`), permitiendo identificar visualmente cada nodo a través del proxy inverso Caddy.

9.4. Docker (docker.nix)

El módulo `docker.nix` habilita el demonio Docker en los hosts que lo requieren mediante `virtualisation.docker.enable = true`, configura el usuario `docker` con los permisos necesarios y define los contenedores que deben arrancarse automáticamente en el boot del sistema mediante unidades `systemd` que invocan `docker compose up`. Los archivos `compose.yml` de cada servicio se despliegan en `/srv/docker/<servicio>/` y los secretos necesarios se montan desde `/run/agenix/` en modo solo lectura.

9.5. Monitorización: Prometheus, Node Exporter y Grafana

La máquina `monitoring` centraliza la observabilidad de toda la infraestructura mediante tres módulos:

- **node_exporter.nix**: habilitado en *todos* los hosts, expone métricas del sistema (CPU, memoria, disco, red) en el puerto `9100` mediante `services.prometheus.exporters.node`.
- **prometheus.nix**: desplegado únicamente en `monitoring`, configura `services.prometheus` con un `scrapeConfig` que apunta a los exporters de cada worker y del propio host Caddy. El intervalo de scraping y las reglas de retención se declaran en este módulo.
- **grafana.nix**: también exclusivo de `monitoring`, levanta `services.grafana` con Prometheus como datasource provisionado automáticamente. Los dashboards se importan como JSON y se exponen al exterior a través del proxy inverso Caddy.

Esta arquitectura permite visualizar en tiempo real el estado de toda la granja de servidores desde un único panel sin necesidad de configuración manual una vez desplegada la infraestructura.

10. Ventajas y limitaciones

10.1. Ventajas

- **Reproducibilidad total**: Podemos recrear toda la infraestructura desde cero en cuestión de minutos usando únicamente los archivos del repositorio `git`.
- **Seguridad**: Gracias a `agenix`, los secretos (tokens de Cloudflare, contraseñas de Docker) viajan cifrados en el repositorio y solo se descifran en el tiempo de ejecución en la memoria RAM de la VM.
- **Gestión unificada**: La combinación de NixOS y Colmena permite gestionar una granja de servidores como si fuera una sola entidad.

10.2. Limitaciones y desafíos

- **Curva de aprendizaje**: El ecosistema Nix (Flakes, lenguaje Nix) tiene una curva de aprendizaje pronunciada comparado con métodos tradicionales de administración.
- **Calidad del Provider**: El proveedor de Proxmox para OpenTofu/Terraform presenta limitaciones en la gestión de estados complejos. En ocasiones, cambios menores en la configuración de hardware provocan largas esperas para que las VM's vuelvan a estar disponibles, esto es acendutado por el hardware que empleamos (los discos duros y controladora principalmente).
- **Automatización incompleta**: No hemos logrado una "automatización de un solo clic" (*full automation*) total; todavía se requiere la creación manual de la API Key en la interfaz de Proxmox y la captura manual del `hardware-configuration.nix` antes del primer despliegue.